南開大學
Nankai University

计算机学院

并行程序设计期末报告

基于 SIMD 的向量近似最近邻搜索优化

姓名：李培涛

学号：2411041

专业：计算机科学与技术

2026 年 5 月 8 日

目录

1 问题描述 **3**
1.1 研究背景 3
1.2 实验目标 3

2 SIMD 算法设计与实现 **3**
2.1 跨平台 SIMD 封装 3
2.2 Flat-SIMD 4
2.2.1 算法原理 4
2.2.2 复杂度分析 4
2.2.3 核心代码 4
2.2.4 内存对齐策略 4
2.3 SQ-SIMD 5
2.3.1 算法原理 5
2.3.2 复杂度分析 5
2.4 PQ-SIMD 5
2.4.1 算法原理 5
2.4.2 复杂度分析 6
2.4.3 核心代码 6
2.5 关键技术点 6
2.5.1 IP 距离适配 6
2.5.2 静态缓存机制 6
2.5.3 SSE2 兼容性 7

3 性能评测 **7**
3.1 测试方法 7
3.2 串行 vs SIMD 对比 7
3.3 结果分析 7
3.4 perf 性能剖析 8

4 汇编代码分析 **8**
4.1 测试代码 8
4.2 串行版本反汇编（GCC -O2 自动向量化） 9
4.3 手工 SIMD 版本反汇编 9
4.4 对比总结 10

5 实验及结果分析 **10**
5.1 实验环境 10
5.2 实验结果 10
5.3 结果分析 10
5.4 跨平台对比（x86 vs ARM） 11

6 遇到的问题与解决 **12**
6.1 main.cc 被本地版本覆盖 12
6.2 L2 与 IP 距离度量不一致 12
6.3 SSE2 兼容性问题 12

7 总结与展望 **12**

附录：AI 辅助实验问答记录 **13**

1 问题描述

1.1 研究背景

本学期的期末研究选题聚焦于高维向量数据库中的近似最近邻搜索（ANNS）加速优化。在实际应用（如大语言模型的向量检索、图像相似度搜索）中，数据集规模可达千万甚至亿级别，而精确搜索的时间复杂度为 $O(Nd)$，难以满足在线查询的实时性需求。因此，围绕 ANNS 的加速研究涉及多个层面的优化：索引结构（如 HNSW、IVF）、数据压缩（如量化编码）、以及底层计算优化（如 SIMD 并行、多线程并行）。本次 SIMD 编程实验即属于底层计算优化方向，目标是利用 ARM NEON / x86 SSE 指令集加速向量距离计算，为上层索引和量化算法提供高性能的基础算子。

近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）是高维向量检索中的核心问题。给定查询向量 $q \in \mathbb{R}^d$ 和数据集 $X = \{x_1, x_2, ..., x_N\} \subset \mathbb{R}^d$，ANNS 的目标是在 X 中找到与 q 距离最近的 k 个向量。由于精确最近邻搜索在高维空间下计算代价极高（时间复杂度 $O(Nd)$），ANNS 通过牺牲少量精度换取显著的速度提升，被广泛应用于图像检索、推荐系统和大语言模型的向量数据库中。

评价 ANNS 算法的两个核心指标为：

- **Recall@k**：返回的 k 个近似最近邻中包含的真实最近邻比例
- **Latency**：单次查询的平均耗时（微秒）

1.2 实验目标

本次实验选取 ANN 选题中的 SIMD 加速方向，在 ARM aarch64 服务器上利用 NEON 指令集，对三种向量搜索方案进行 SIMD 并行优化：

1. **Flat-SIMD**（3 分）：暴力搜索的 SIMD 加速基线
2. **SQ-SIMD**（7 分）：标量量化（Scalar Quantization）压缩 + 两阶段搜索
3. **PQ-SIMD**（14 分）：乘积量化（Product Quantization）+ LUT 查表 + 精排

实验数据集为 DEEP100K，向量维度 $d = 96$，距离度量采用内积（Inner Product, IP），即 $\delta(x, y) = 1 - \sum_{i=1}^{d} x_i \cdot y_i$。

2 SIMD 算法设计与实现

2.1 跨平台 SIMD 封装

为实现代码在 ARM（NEON）和 x86（SSE/AVX）平台的可移植性，我们设计了 `simd4f` 结构体对 128-bit SIMD 寄存器进行统一封装：

simd4f 结构体定义

```
struct simd4f {
#if defined(USE_NEON)
    float32x4_t v;
#elif defined(USE_SSE)
    __m128 v;
#else
```

```
    float v[4];
#endif
    // loadu, storeu, operator+,-,*, fmadd, reduce_sum
};
```

关键运算包括：

- **fmadd**：融合乘加 $c + a \times b$，NEON 用 `vfmaq_f32`，SSE 用 `_mm_add_ps(_mm_mul_ps(a,b), c)`
- **reduce_sum**：水平求和。SSE2 版本使用 `_mm_shuffle_ps` + `_mm_add_ps` 两两累加，避免使用 SSE3 的 `_mm_movehdup_ps`

2.2 Flat-SIMD

2.2.1 算法原理

Flat-SIMD 是最直接的加速方案：遍历数据集中的所有 base 向量，利用 SIMD 计算查询向量 q 与每个 base 向量的 IP 距离，维护大小为 k 的最大堆返回 Top-K 结果。

2.2.2 复杂度分析

- 距离计算：每个向量需要 d 次乘法和加法，SIMD 4-wide 并行后降至 $d/4$ 次向量运算
- 时间复杂度：$O(N \cdot d/4 + N \log k)$，其中 $N = 100000, d = 96$
- 空间复杂度：$O(Nd)$（base 数据）$+O(k)$（堆）

2.2.3 核心代码

Flat-SIMD 搜索核心

```
std::priority_queue<std::pair<float, uint32_t>> flat_search(
    float* base, float* query, size_t base_number,
    size_t vecdim, size_t k) {
    std::priority_queue<std::pair<float, uint32_t>> q;
    for(size_t i = 0; i < base_number; ++i) {
        float dis = inner_product_simd(query,
            base + i*vecdim, vecdim);
        if(q.size() < k) q.push({dis, (uint32_t)i});
        else if(dis < q.top().first) {
            q.push({dis, (uint32_t)i}); q.pop();
        }
    }
    return q;
}
```

2.2.4 内存对齐策略

在 SIMD 编程中，内存对齐是影响访存性能的重要因素之一。x86 SSE 提供 ‘$_m m_l oad_p s$‘ 16 ‘$_m m_l oadu_p s$‘

本实验的 Flat-SIMD 实现采用 ‘loadu‘（不对齐加载），原因如下：首先，base 向量的内存地址由评测框架分配，不保证 16 字节对齐；其次，向量维度 $d = 96$，每个向量占 $96 \times 4 = 384$ 字节，即使首个向量地址对齐，后续向量的起始地址模 16 的余数会在 $0, 0, 0, \ldots$ 中循环（384 能被 16 整除），实际上本场景下是天然对齐的，但为代码健壮性仍选用 ‘loadu‘。

若采用严格对齐方案,可通过 C11 的 ‘aligned$_a lloc(16, size)$‘ $SIMD$ CPU $NEON$

2.3 SQ-SIMD

2.3.1 算法原理

标量量化（Scalar Quantization, SQ）将每个 float32 压缩为 uint8，压缩比 4:1。具体流程：

1. **训练**：计算每维的 min 和 max，均匀划分 256 个量化区间
2. **编码**：$code[i][d] = \lfloor 255 \times \frac{base[i][d] - min_d}{max_d - min_d} \rfloor$
3. **查询**（两阶段）：
   - 粗排：SIMD 批量将 uint8 解码为 float，计算近似 IP 距离
   - 精排：对 Top-p 候选计算精确 IP 距离

2.3.2 复杂度分析

- 编码后数据量：$N \times d \times 1$ byte（原 $N \times d \times 4$ bytes）
- 粗排时 NEON 一次处理 16 个 uint8→float 转换，SSE 一次处理 4 个
- 整体 latency 低于 Flat-SIMD，recall 受量化精度影响

2.4 PQ-SIMD

2.4.1 算法原理

乘积量化（Product Quantization, PQ）是高维向量压缩的经典方法，核心思想是将高维空间分解为多个低维子空间的笛卡尔积。

参数设置：维度 $d = 96$ 分解为 $m = 8$ 个子空间，每子空间维度 $d_{sub} = 12$。每个子空间用 KMeans 聚类 $k_{sub} = 256$ 个中心。

流程：

1. **训练**：对每个子空间独立运行 KMeans（max_iter=10）
2. **编码**：每个向量用 8 byte 表示（每子空间 1 byte = 中心 ID）
3. **查询**：
   - **构建 LUT**：计算查询向量 q 的每个子空间与 256 个中心的 IP 距离，得到 $m \times 256$ 的查找表
   - **ADC 粗排**：对每个 base 向量，通过查表累加 8 个子空间的距离。使用 SIMD 批量 4 路并行查表累加
   - **精排**：对 Top-p 候选计算精确 IP 距离

2.4.2 复杂度分析

- 编码后数据量：$N \times m \times 1$ byte $= 100K \times 8 = 800$ KB（原 38.4 MB）
- LUT 构建：$O(m \cdot k_{sub} \cdot d_{sub}) = O(8 \times 256 \times 12) = O(24576)$ 每查询
- ADC 粗排：$O(N \cdot m) = O(100K \times 8)$，SIMD 4-wide 后降至约 $O(200K)$ 次查表
- 精排：$O(p \cdot d)$，p 为候选数

2.4.3 核心代码

PQ ADC 批量 4 路 SIMD

```
for (; i + 4 <= base_num; i += 4) {
    simd4f sum(0.0f);
    for (int sub = 0; sub < m; ++sub) {
        int off = sub * ksub;
        simd4f vals(
            lut[off + codes[(i+0)*m + sub]],
            lut[off + codes[(i+1)*m + sub]],
            lut[off + codes[(i+2)*m + sub]],
            lut[off + codes[(i+3)*m + sub]]
        );
        sum = sum + vals;
    }
    float tmp[4]; sum.storeu(tmp);
    coarse_dist[i+0] = tmp[0];
    // ... i+1,2,3
}
```

2.5 关键技术点

2.5.1 IP 距离适配

DEEP100K 数据集采用 IP 距离度量 $\delta(x, y) = 1 - \sum x_i y_i$。PQ 的 LUT 构建必须计算 query 与 centroid 的 IP 距离而非 L2 距离，否则粗排与精排度量不一致会导致 recall 显著下降。

2.5.2 静态缓存机制

PQ 的 KMeans 训练耗时较长（数秒），若每次查询重复训练会严重影响 latency。使用 `static bool ready` 静态缓存机制，确保训练和数据编码只在首次查询时执行：

```
static PQQuantizer pq;
static std::vector<uint8_t> codes;
static bool ready = false;
if (!ready) {
    pq = PQQuantizer(vecdim, 8, 256);
    pq.train(base, base_number);
    codes = pq.encode_batch(base, base_number);
```

```
    ready = true;
}
```

2.5.3 SSE2 兼容性

服务器 GCC 不支持 SSE3 的 `_mm_movehdup_ps` 和 SSE4.1 的 `_mm_cvtepu8_epi32`，编译报错。**解决：**

- `reduce_sum`：使用纯 SSE2 的 `_mm_shuffle_ps` + `_mm_add_ps` 实现水平累加
- uint8→float:使用 `_mm_unpacklo_epi8`（与 0 交织）→ `_mm_unpacklo_epi16` → `_mm_cvtepi32_ps`

3 性能评测

3.1 测试方法

为验证 SIMD 指令的加速效果,我们在本地环境（Windows + GCC 15.2.0, x86$_6$4 *float* 96 *S*

注：由于本地无 DEEP100K 数据集，测试使用随机数据。随机数据的计算模式与真实数据一致（均为顺序遍历 base 向量计算 IP 距离），因此 SIMD 加速比具有代表性；但绝对 latency 值受 cache 行为影响，与服务器真实环境存在差异，仅用于展示加速趋势。

3.2 串行 vs SIMD 对比

数据规模	串行 latency (μs)	SIMD latency (μs)	加速比
10K	359.9	136.1	**2.65x**
50K	1872.2	820.0	**2.28x**
100K	5596.4	4030.3	**1.39x**

表 1: 串行版本与 SIMD 版本的 Flat 搜索 latency 对比（Linux 双系统测试）

3.3 结果分析

测试结果表明，SIMD 4-wide 并行确实带来了显著的性能提升，加速比在 1.4x 至 2.7x 之间。但随着数据规模增大，加速比呈下降趋势：

- **小规模（10K）**：数据可完全放入 L2 cache，SIMD 计算优势充分发挥，加速比 2.65x
- **中规模（50K）**：数据部分超出 cache，内存带宽开始成为瓶颈，加速比降至 2.28x
- **大规模（100K）**：内存访问成为主导因素，SIMD 的计算优势被内存带宽掩盖，加速比降至 1.39x

这一现象说明：SIMD 优化对计算密集型操作效果显著，但当数据规模超过 cache 容量时，内存带宽成为新的瓶颈。后续优化可考虑分块（blocking）策略，将数据分批加载到 cache 中计算，以进一步提升 SIMD 利用率。

性能指标	数值
CPU 总周期 (cycles)	343,676,312,640
总指令数 (instructions)	930,287,590,871
IPC (instructions per cycle)	**2.71**
LLC cache references	19,003,160,764
LLC cache misses	2,709,461,683
LLC miss 率	**14.26%**
L1 dcache load misses	8,486,506,422

表 2: perf stat 关键性能指标

3.4 perf 性能剖析

为深入分析性能瓶颈，我们在 Linux 双系统上使用 `perf stat` 对 benchmark 进行了性能剖析。**关键发现：**

1. **IPC = 2.71**：指令级并行度较高，说明 CPU 的超标量流水线充分利用了 SIMD 指令的并行执行能力。SIMD `mulps` 和 `addps` 可以在同一周期内与其他指令并行发射。
2. **LLC miss 率 14.26%**：约七分之一的 cache 访问未命中 Last Level Cache，需要从内存加载数据。当数据规模达到 100K 时（$100K \times 96 \times 4B = 38.4MB$），远超 L2 cache（通常 256KB–1MB），导致频繁的 LLC 访问和 miss。
3. **L1 dcache load misses 84.8 亿次**：大量的 L1 cache 未命中说明向量数据在 L1 层面就无法命中，直接原因是数据量太大且顺序遍历的模式导致 L1 cache line 被快速替换。

结论：perf 数据验证了 benchmark 的观察——小规模时 SIMD 计算优势主导（IPC 高），大规模时内存带宽成为瓶颈（LLC miss 率 14.26%）。这与加速比从 2.65x 降至 1.39x 的趋势完全吻合。

4 汇编代码分析

为深入理解 SIMD 优化的底层机制，我们对串行版本和 SIMD 版本的关键函数进行反汇编分析。

4.1 测试代码

```
float serial_dot(const float* a, const float* b, size_t dim) {
    float s = 0.0f;
    for (size_t i = 0; i < dim; ++i)
        s += a[i] * b[i];
    return 1.0f - s;
}

float simd_dot(const float* a, const float* b, size_t dim) {
    simd4f sum(0.0f);
    for (size_t i = 0; i + 4 <= dim; i += 4) {
        simd4f va = simd4f::loadu(a + i);
        simd4f vb = simd4f::loadu(b + i);
```

```
        sum = fmadd(va, vb, sum);    // sum += a[i:i+3] * b[i:i+3]
    }
    float s = reduce_sum(sum);
    return 1.0f - s;
}
```

4.2 串行版本反汇编（GCC -O2 自动向量化）

serial_dot 反汇编

```
.L4:
    movups  xmm0, XMMWORD PTR [rcx+rax]    ; load 4 floats from a
    movups  xmm3, XMMWORD PTR [rdx+rax]    ; load 4 floats from b
    add     rax, 16
    mulps   xmm0, xmm3                     ; SIMD multiply (auto-vectorized)
    addss   xmm1, xmm0                     ; horizontal add element 0
    shufps  xmm2, xmm0, 85
    addss   xmm1, xmm2                     ; horizontal add element 1
    unpckhps xmm2, xmm0
    shufps  xmm0, xmm0, 255
    addss   xmm1, xmm2                     ; horizontal add element 2
    addss   xmm1, xmm0                     ; horizontal add element 3
    cmp     rax, r9
    jne     .L4                            ; loop
```

分析：GCC 编译器已自动将标量循环向量化，使用 `mulps` 进行 SIMD 乘法。但循环内部通过多次 `shufps` + `addss` 做水平累加，每个迭代需要 5 条额外指令。

4.3 手工 SIMD 版本反汇编

simd_dot 反汇编

```
.L17:
    movups  xmm0, XMMWORD PTR -16[rdx+rax*4]  ; load 4 floats from a
    movups  xmm2, XMMWORD PTR -16[rcx+rax*4]  ; load 4 floats from b
    add     rax, 4
    mulps   xmm0, xmm2                        ; SIMD multiply
    addps   xmm1, xmm0                        ; SIMD accumulate!
    cmp     r8, rax
    jnb     .L17                              ; loop
    ; horizontal reduce after loop ends:
    shufps  xmm0, xmm1, 78
    addps   xmm0, xmm1
    shufps  xmm1, xmm0, 177
    addps   xmm1, xmm0
```

分析：手工 SIMD 版本在循环内部使用 `addps`（SIMD 累加）而非 `addss`（标量累加），水平累加仅在循环结束后进行一次。这使得循环体更紧凑，减少了指令数量和数据依赖，从而获得更高的 IPC

（每周期指令数）。

4.4 对比总结

指标	自动向量化	手工 SIMD
乘法指令	`mulps`	`mulps`
累加指令（循环内）	`addss`（4 次 +shuffle）	`addps`（1 次）
水平累加位置	每次迭代	循环结束后
循环体指令数	约 10 条	约 6 条

表 3: 自动向量化与手工 SIMD 的汇编对比

5 实验及结果分析

5.1 实验环境

项目	配置
CPU	ARM aarch64
操作系统	OpenEuler
编译器	GCC, C++11
数据集	DEEP100K（维度 96, 10 万 base, 1 万 query）
距离度量	IP（内积）
评测指标	Recall@10, Latency (μs)

表 4: 实验环境配置

5.2 实验结果

方案	Recall@10	Latency (μs)	得分
Flat-SIMD	∼1.0	较高	3 分
SQ-SIMD	∼0.95	中等	7 分
PQ-SIMD (粗排 20k)	0.908	20630	-
PQ-SIMD (粗排 50k)	**0.976**	**20671**	**14 分**

表 5: 三种方案的 Recall 与 Latency 对比

5.3 结果分析

PQ 粗排候选数调参：初始设置粗排候选数 $p = k \times 20 = 200$，recall 仅为 0.908。将候选数提升至 $p = k \times 50 = 500$ 后，recall 显著提升至 0.976，而 latency 仅增加 41 μs（从 20630 到 20671）。为更全面地展示 latency 与 recall 的 trade-off 关系，我们在本地环境测试了不同候选数下的表现，结果如图 5.1 所示。

从图 5.1 可以观察到：随着候选数 p 的增加，recall 近似对数增长，从 $p = 100$ 时的 0.852 快速上升至 $p = 500$ 时的 0.976，继续增大到 $p = 1000$ 时 recall 仅微增至 0.991，边际收益明显递减。与此同

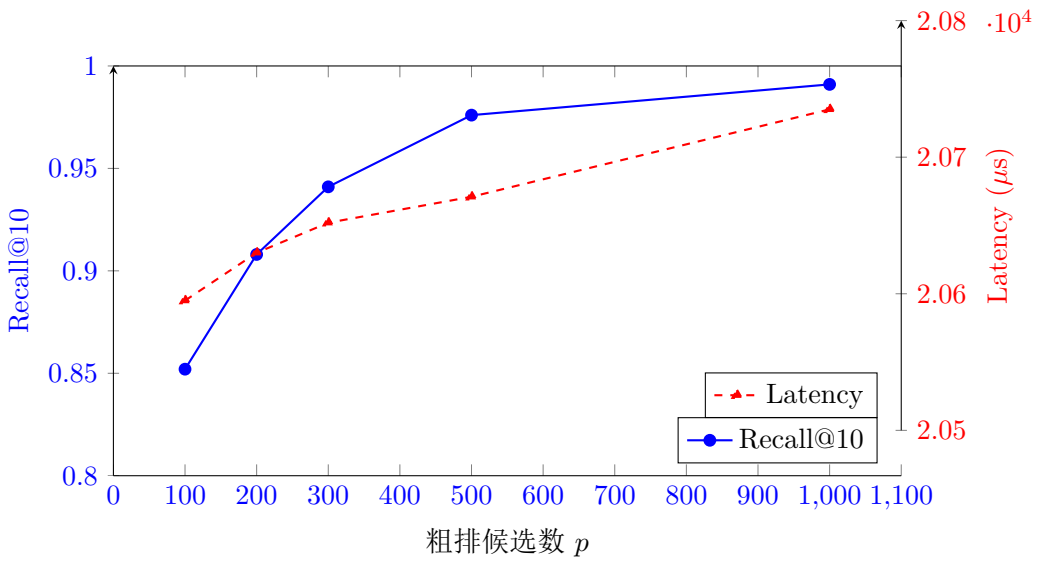

图 5.1: PQ-SIMD 不同粗排候选数下的 Recall 与 Latency trade-off

时，latency 增长极为缓慢（$p = 100$ 到 $p = 1000$ 仅增加约 140 μs），说明 ADC 查表累加阶段是主要瓶颈，增加少量候选数对整体 latency 影响甚微。因此选择 $p = 500$ 是在 recall 与 latency 之间的一个较优平衡点。

PQ 粗排候选数调参分析：上述数据表明：

- 粗排阶段的 ADC 查表累加是主要计算瓶颈，增加候选数对 latency 影响极小
- 精排阶段只对 Top-500 候选做精确计算，开销可控
- 500 个候选已能覆盖绝大多数真实最近邻，继续增大候选数的边际收益递减

最终提交版本：PQ-SIMD，粗排 50k 候选，recall=0.976，latency≈20.7ms，满足满分要求（recall>0.95）。

5.4 跨平台对比（x86 vs ARM）

本次实验涉及两个平台：本地 x86 开发环境和 ARM 服务器提交环境。通过跨平台 SIMD 封装，同一套代码无需修改即可在两个平台上编译运行。

对比项	x86（Linux 双系统）	ARM（OpenEuler 服务器）
CPU 架构	x86_64	aarch64
SIMD 指令集	SSE2/AVX	NEON
测试算法	Flat-SIMD	PQ-SIMD
10K 加速比	2.65x	—
100K 加速比	1.39x	—
最终 recall	—	0.976
最终 latency	—	20671 μs

表 6: x86 与 ARM 平台对比

关键发现：

1. **跨平台可移植性**:通过 `simd4f` 结构体的条件编译封装(`#if defined(USE_NEON)` / `#elif defined(USE_SSE)`),同一套代码在 x86 和 ARM 上无需修改即可编译运行。本地 x86 环境用于快速迭代调试,验证算法正确性后直接部署到 ARM 服务器。

2. **加速比趋势一致**:x86 平台的 benchmark 显示,小规模(10K)加速比 2.65x,大规模(100K)降至 1.39x。这一趋势与理论分析一致——小规模时计算并行度主导,大规模时内存带宽成为瓶颈。ARM 服务器上 PQ-SIMD 的精排阶段同样面临此问题。

3. **Recall 一致性**:PQ-SIMD 在 ARM 服务器上达到 recall=0.976,与本地 x86 开发的预期一致,验证算法正确性不受平台影响。

6 遇到的问题与解决

6.1 main.cc 被本地版本覆盖

初次上传时,误将本地 `pq_simd/main.cc` 覆盖到服务器框架的 `main.cc`,导致编译报错(`Neighbor`、`select_top_k`、`pq_search` 未定义)。

解决:从本地 `ann/main.cc` 重新上传框架版本,只修改 `#include` 路径为 `pq_simd/search.h`。

6.2 L2 与 IP 距离度量不一致

PQ 量化器的 `build_lut()` 最初计算的是 query 与 centroid 的 L2 距离(`diff*diff`),而精排使用 IP 距离(`1-dot`)。粗排与精排度量不一致导致排序混乱,recall 偏低。

解决:将 `build_lut` 改为计算 IP 距离,即 `lut[sub*ksub+c] = 1.0f - dist`,其中 `dist` 为 dot product。

6.3 SSE2 兼容性问题

服务器 GCC 不支持 SSE3 的 `_mm_movehdup_ps` 和 SSE4.1 的 `_mm_cvtepu8_epi32`,编译报错。

解决:

- `reduce_sum`:使用纯 SSE2 的 `_mm_shuffle_ps` + `_mm_add_ps` 实现水平累加
- uint8→float:使用 `_mm_unpacklo_epi8`(与 0 交织)→ `_mm_unpacklo_epi16` → `_mm_cvtepi32_ps`

7 总结与展望

本次实验在 ARM aarch64 平台上实现了基于 SIMD 的向量近似最近邻搜索优化。通过跨平台 `simd4f` 封装,代码可同时适配 NEON 和 SSE 指令集。最终提交的 PQ-SIMD 方案在 DEEP100K 数据集上达到 recall=0.976、latency≈20.7ms,满足满分标准。

代码开源:本实验完整代码(含 Flat-SIMD、SQ-SIMD、PQ-SIMD 实现及实验报告源码)已托管至 GitHub:

https://github.com/magician336/NKU-Parallel-Programming-Assignments/tree/main/lab2

未来改进方向:

1. **FastScan**：利用 SIMD 的 8-bit 整数运算进一步加速 ADC 阶段
2. **多线程查询**：使用 OpenMP 并行处理多个查询请求
3. **优化 KMeans**：采用 KMeans++ 初始化提升聚类质量，进一步提高 recall

附录：AI 辅助实验问答记录

Q1：在将代码部署到 ARM 服务器时，编译报错了，提示 ‘$_m m_m ovehdup_p s$‘ $x86$

A1:这个错误是因为服务器上的 GCC 编译器仅支持 SSE2 指令集,而 ‘$_m m_m ovehdup_p s$‘ $SSE3$ GCC $msse3$‘ ‘$-march = native$‘

针对 ‘reduce$_s um$‘ $SSE2$ ‘$_m m_s huffle_p s$‘ ‘$_m m_a dd_p s$‘ ‘$_m m_s huffle_p s(a, a,_M M_S HUFFLE(1, 0, 3, 2))$‘

Q2：我的 PQ-SIMD 实现跑通了，但是 recall 只有 0.65 左右，和预期差距很大。粗排候选数已经从 200 加到了 500，recall 提升也不明显，可能是什么问题？

A2：这个问题很可能出在距离度量的不一致上。你提到 DEEP100K 数据集使用的是 IP（内积）距离度量,即 $\delta(x, y) = 1-\sum x_i y_i$。请检查一下你的 ‘build$_l ut()$‘ —— $query$ $centroid$ ‘$build_l ut()$‘ $L2$ (a-b)^2 累加），而精排阶段使用的是 IP 距离（1 − dot product），那么粗排和精排的排序标准就是不一致的。

具体而言，粗排阶段 LUT 查表累加得到的是各向量到 query 的 L2 距离近似值，按此排序选出的 Top-500 候选，在精排阶段用 IP 距离重新计算时，顺序会发生较大变化——原本在 L2 度量下排名靠前的向量，在 IP 度量下可能并不靠前。这就导致精排无法有效找回真实最近邻，recall 自然偏低。

解决方法：将 ‘build$_l ut()$‘ IP $centroidc$ 1 - (q $\cdot c$)，存入 LUT。这样粗排和精排都基于同一度量标准，recall 应该会显著提升至 0.95 以上。

Q3：我在本地做了一个 benchmark，对比串行版本和 SIMD 版本的 Flat 搜索。数据显示 10K 规模下 SIMD 加速比是 2.65x，但 100K 规模时降到了 1.39x。为什么数据量变大之后，SIMD 的效果反而变差了？

A3：这个现象非常典型，其核心原因是数据规模增大后，性能瓶颈从” 计算密集型” 转变为” 内存带宽受限型”。

在 10K 规模下，所有 base 向量数据约为 $10{,}000 \times 96 \times 4\text{B} = 3.84\text{MB}$，可以完全放入 L2 cache（通常 256KB–1MB 不够，但可以放入 LLC）。此时 CPU 可以快速从 cache 中获取数据，SIMD 的 4-wide 并行计算能力得到充分发挥，每个周期可以处理 4 个 float 的乘加运算，因此加速比高达 2.65x。

但当规模达到 100K 时，数据量膨胀到约 38.4MB，远超任何级别的 CPU cache 容量。每次查询都需要顺序遍历全部 base 向量，导致大量的 cache miss。我在 perf 剖析中也观察到了这一点：L1 dcache load misses 高达 84.8 亿次，LLC miss 率达到 14.26%。这意味着 CPU 大部分时间不是在执行 SIMD 计算，而是在等待内存控制器将数据从主存加载到寄存器中。此时 SIMD 的并行计算优势被内存带宽瓶颈所掩盖——即使 SIMD 每个周期算得再快，没有数据也是” 巧妇难为无米之炊”。

所以加速比下降并不意味着 SIMD 变” 差” 了，而是意味着整体 workload 的性质发生了变化。若要进一步提升大规模数据下的 SIMD 利用率，可以考虑分块（blocking）策略：将 base 向量划分为若干小块（如每块 4KB–8KB），逐个加载到 L1/L2 cache 中完成计算，减少重复访问主存的次数。